

LLMTuner: An Interactive System for Behavioral Fine-Tuning of Language Models Through Human Feedback

Mandy Reyes

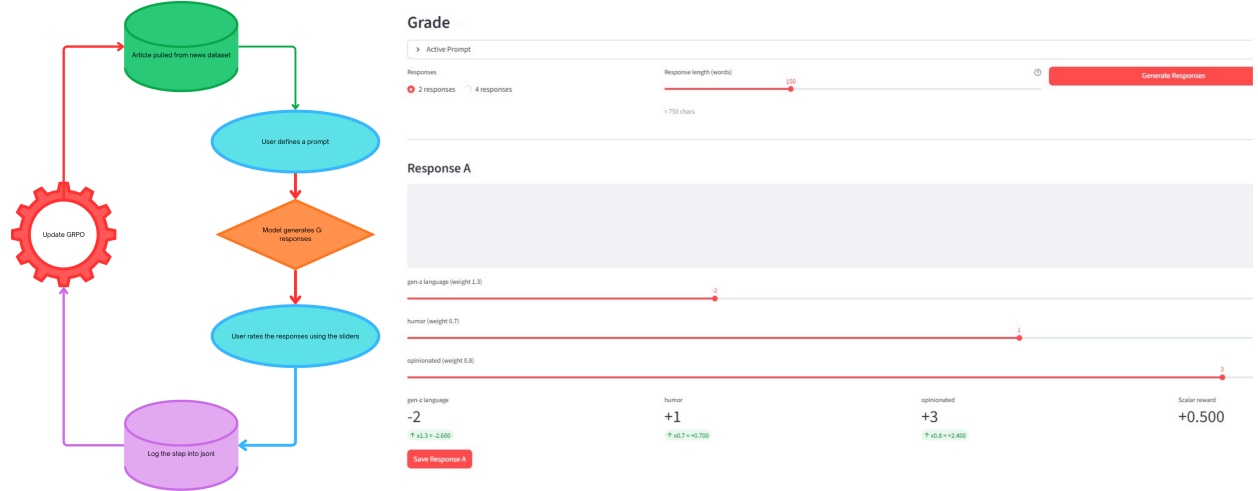


Fig. 1: LLMTuner end-to-end pipeline. The user defines behavioral traits, selects a 20 Newsgroups article, generates and grades model responses, and triggers online GRPO training, all through a no-code Streamlit interface running entirely on consumer GPU hardware.

Abstract— Large language models have been dominating every headline and have become incredibly accessible for any developer looking to work with them. However, adapting them to specific behavioural objectives still serves to be a daunting and computationally expensive task for most users. There are already existing reinforcement learning with human feedback pipelines that exist but the requirements needed to run those systems far exceed what the average user possesses. More specifically, the background technical knowledge and the GPU infrastructure needed to host such pipelines limit the accessibility to fine tune LLMs interactively to those with machine learning experience and research environments or access to cloud computing environments. I propose an interactive system for RLHF that allows users to fine tune a language model toward user-defined behavioral objectives through direct slider-based feedback, without the need to write any code. The system combines 4-bit QLoRA quantization and LoRA adapters to run entirely on a consumer GPU with as little as 4GB of VRAM, and implements a custom online Group Relative Policy Optimization algorithm. Experiments show that reward trends remain stagnant when fine tuning objectives overlap with the base model's existing capabilities, but show improvement when objectives are specific enough that the base model does satisfy them naturally. These results show that a RLHF GPRO system can produce meaningful results on consumer hardware.

1 INTRODUCTION

The recent surge in artificial intelligence and its incredible achievements has made large language models and chatbots all the more appealing to users. Automating and delegating tasks to the aforementioned language models have been increasingly common. It is fairly simple to log on to a chatbot account and open one of these tools. Yet, such chatbots have broad training parameters and usages, making it difficult to specify the responses to give out exactly what the user actually wants. Finetuning models is a popular way to tailor models to specific results, but they often need large amounts of data to be able to have an impact on the model weights, many times, this approach ends up producing a subpar trained model. Through reinforcement learning [1], we can train models in real time by giving it feedback as soon as the response is generated, this way, the model can know whether the response it generated aligns with the user's ideals or not. I hypothesize that repeating this step over and over will result in a model that responds in a way the user is content with. However, fine tuning and training these large language models can

be a tough task even for those with a technical background. I present an interactive system for reinforcement learning with human feedback tailored to finetune LLMs to a user's specific needs. The system tackles two of the main problems presented earlier. Firstly, by fine tuning the model to the user's specifics we narrow the scope of the tasks that the LLMs respond to. Secondly, we abstract the technical implementations, the user only needs to write prompts and grade responses through interactive sliders, allowing the model to be trained without writing a single line of code.

1.1 Model and Dataset

The model used in this system is Qwen2.5-1.5B-Instruct [7], a compact instruction-tuned language model well-suited for interactive fine-tuning since it is relatively small and is already tuned to follow the user's instructions. Fine-tuning is performed using LoRA [3] through HuggingFace's PEFT library, which freezes the base model weights and trains only a small set of low-rank adapter matrices, reducing the number of trainable parameters to approximately 0.3% of the full model. Combined with 4-bit QLoRA quantization [2], the system runs entirely on consumer hardware without requiring a heavier GPU. To provide a supplemental corpus for the response generation process, the system uses articles from the 20 Newsgroups dataset [4], a standard benchmark corpus of approximately 18,000 articles across twenty categories. Articles are preprocessed to remove headers, footers, and miscellaneous content before being given to the users.

1.2 Contributions

Most general RLHF pipelines require large and heavy software, offline data collection, a second reference model with the original model weights to use as a baseline for GRPO [8] training. However, since I cannot guarantee that every user who works with the system (and even myself) would meet the hardware expectations, I design a way for the system to be runnable locally on laptops with a discrete GPU of at least 4GB VRAM. Moreover, standard GRPO requires a pre-defined reward function and collects a full batch of completions before any gradient update, which does not support the interactive, per-session reward grading this system is designed around. As a result, I adapted the GRPO training workflow into an interactive human-in-the-loop system that retrieves feedback directly from the UI and computes rewards from user-provided scores. GRPO also uses a KL divergence penalty to stop the model from drifting too far into reward hacking and spewing out gibberish responses, however, having a second copy of the original weights takes up a substantial amount of memory. Notably, LoRA’s adapter-based architecture enables the same model weights to serve as both the policy and the frozen reference, by toggling adapters on and off during the forward pass, eliminating the need for a second model copy entirely. The human user’s responsibility is to steer the model using an appended prompt, rate responses that the model generates using interactive sliders. The GRPO training step runs every three rounds of responses, this gives the reward signals enough leeway to not be heavily affected by a single noisy reward. The full system is run on a streamlit UI that integrates all of the tools in the system into a usable session and when a user is content with the model’s responses the model or model weights can be exported. Lastly, this all runs comfortably in a 4GB GPU. The remainder of this paper is organized as follows: Section 2 reviews related work in reinforcement learning from human feedback, parameter-efficient fine-tuning, and existing RLHF tooling. Section 3 outlines the design goals and tasks that motivated the system’s development. Section 4 describes the system design, covering the architecture, reward system, session management, and VRAM budget. Section 5 details the custom GRPO algorithm, including the training setup and online step implementation. Section 6 presents the user interface and its six pages. Section 7 describes use cases the system supports. Section 8 discusses results, limitations, and future work, and finishes off with a conclusion in Section 9.

2 RELATED WORKS

In this section, I review related work in reinforcement learning from human feedback, group relative policy optimization, parameter-efficient fine-tuning, and interactive RLHF tooling. I introduce key concepts and identify some gaps that become motivation and added implementation in my system and contributions.

2.1 Reinforcement Learning from Human Feedback

The RLHF pipeline was formalized by Ouyang et al. [6] in their InstructGPT paper, which demonstrated that large language models are able to be aligned to human preferences through a three step pipeline: supervised fine tuning on data, reward model training on human preference judgements, and policy optimization using Proximal Policy Optimization. This became the standard pipeline for aligning LLMs. However, this methodology requires a large volume of preference data, a separately trained reward model, and GPU power that exceeds most of the hardware for my target users. Instead, my system addresses these hurdles and limitations by replacing the reward model with direct human scoring through a trait-weighted slider interface, replacing PPO with GRPO to eliminate the separate reward neural network, and replacing the compute requirements with a consumer hardware deployment through QLoRA. This allows the original idea of a human-feedback-driven fine tuning while making the system accessible to individual researchers that do not have access to heavy GPU infrastructure.

2.2 Group Relative Policy Optimization

GRPO was introduced by Shao et al. [8] as an efficient alternative to PPO that removes the need for a value network by computing advantages relative to a group of completions for the same prompt. Instead of

estimating a reward baseline from a separately trained neural network, GRPO generates G completions per prompt, scores them all using a reward function, and then normalizes these rewards within a group to produce relative advantages. This reduces the memory needed substantially compared to PPO. My system adopts GRPO as its core training algorithm but also extends it by producing rewards by interactive human scoring through slider inputs rather than an automated verifier and the gradient step is reimplemented from scratch as a function outside of the batch trainer, to support the human feedback collection.

2.3 Parameter-Efficient Fine-Tuning and Quantization

LoRA [3] introduced low-rank adaptation as a method for fine-tuning large language models by inserting trainable low-rank matrices into the frozen base model weights, which reduces the number of trainable parameters while still preserving the capacity of the model. QLoRA [2] extended this approach by combining LoRA with 4-bit NF4 quantization and double quantization, enabling fine-tuning of models that would not otherwise fit within available GPU memory.

2.4 RLHF Tooling

TRL [9] is the most widely used open-source implementation of RLHF training pipelines, including implementations of PPO, DPO, and GRPO through its GRPOTrainer interface. I initially planned on using this library in my implementation of my system but the architecture is designed for offline settings where a reward model or rule-based scorer can evaluate completions without human involvement, but it does not support the integration of human feedback collection with the gradient step that my online RLHF system requires.

3 DESIGN GOALS AND TASKS

3.1 Goals

The overarching goal of the system is to make interactive RLHF accessible to non domain experts working on consumer hardware. This is done through four main goals:

- **G1 — No-code fine-tuning:** The system should allow the user to define behavioral objectives, collect human feedback, and train a language model through the interface. There should be no need for programming knowledge to complete the full RLHF workflow from trait definition to model export.
- **G2 — User-defined reward criteria:** The system should be able to adapt to the rewards defined by the user and produce criteria fully based on the user’s scalar gradings. Users should be able to define, weight, and adjust their own behavioral traits at runtime, making the reward function configurable to their objective without having to manually modify any underlying code. The same system should be able to fine tune the model for any type of set of behaviours.
- **G3 — Immediate feedback on model behavior:** Model updates should be observable within the same session so the user can see the effect of their scoring decisions on subsequent generations without restarting or reloading. Training should not take an exorbitant amount of time.
- **G4 — Consumer hardware:** The system should run on a GPU with as little as 4GB of VRAM, making it accessible to researchers who do not have access to cloud computing or heavy GPU infrastructure. The full loop should be able to execute on a consumer GPU in a reasonable amount of time.

3.2 Tasks

Given these goals, I list a series of tasks that served as a framework to the development of the system:

- **Task 1 — Define reward traits and weights:** The user must be able to specify what behavioral qualities they want to reinforce or suppress, as well as the strength of each quality towards reward signal. Defined traits should be addable, removable, and reweightable at any point during a session without affecting previously saved interactions.

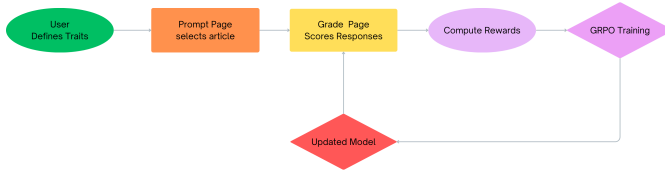


Fig. 2: End-to-end system pipeline. The user defines traits, selects an article, generates responses, scores them with sliders, and triggers an online GRPO update, all within a single Streamlit session.

- **Task 2 — Browse and select training prompts:** The user must be able to browse through a dataset of articles, filter by length and category, preview individual posts, and select one as the basis for the generation prompt.
- **Task 3 — Generate and score model responses:** The user must be able to generate multiple responses to a prompt and score each one independently across all defined traits, with the scalar reward computed and displayed in real time as slider values are adjusted, and each response should receive its own independent score.
- **Task 4 — Trigger and monitor online training:** The system should automatically run the online GRPO training step once enough iterations have been accumulated. The model should be automatically updated and usable once the training has been completed.
- **Task 5 — Resume experiments:** The user must be able to save their training progress, resume it after restarting the system, and run multiple isolated experiments in different sessions.
- **Task 6 — Export the trained model:** Once satisfied with the fine tuned model’s behavior, the user must be able to export the trained model.

4 SYSTEM DESIGN

4.1 Architecture Overview

The system layers each component of the pipeline, starting from the top: a streamlit UI neatly presents each user interaction, it connects the human responses and the online training through a backend adapter. Upon initializing the system, the language model is loaded and cached as a shared resource with streamlit `@st.cache_resource`. This instance is used for both inference and training so that when the training step occurs, it automatically reflects the updated model in the next iteration.

The Qwen2.5-1.5B-Instruct model is finetuned using LoRA configurations, which targets query, key, value, and output projection matrices across attention layers, specifically layers with rank 16 and alpha 32. This drastically reduced the number of trainable parameters from 1.5 billion to approximately 4.4 million, which comes to around 0.3% of the full model. Table ?? summarizes the LoRA configuration used throughout the system. The LoRA B matrices, which are the matrices that are responsible for stretching the compressed low-dimensional signal from the A matrices and learns how much each compressed dimension should nudge each output dimension, are zeroed out upon initialization. On startup, the model behaves identically to the unmodified and untrained base model, and the fine tuning accumulates as the training steps increase.

Training prompts are sourced from the *20newsgroups* dataset using scikit-learn’s `fetch-20newsgroups` that is cleaned up at load time. An instruction template written by the user is appended to the final prompt before passing it on to be wrapped by Qwen2.5’s chat template.

4.2 Reward System

The reward system is designed to combine the user’s configured traits to produce a scalar value that will be sent into the GRPO training. Before beginning the training session, the user defines a set of behavioural traits that each have a name, description, and a numeric weight. These traits represent the qualities that the user wants to reinforce or suppress in the model’s responses.

After the model generates responses for a given prompt, the user scores each response independently on a per-trait basis using sliders that range from -5 to $+5$. Setting a score to 0 indicates that the response is neutral for this trait. Positive scores reward the response and negative responses penalize it. Each response receives a scalar reward computed as a weighted sum across all of the traits:

$$\text{scalar_reward} = \sum (\text{trait.weight} \times \text{slider_score}) \quad (1)$$

This completion scoring is important to the GRPO training step since appending a scalar reward to each of the responses generated, the normalization step can distinguish between responses to the same prompt, rather than treating the entire prompt group as a single signal. All of the interactions are saved into a JSONL file with the prompt, response, per-trait scores, scalar reward, and timestamp, which feeds into the GRPO step directly and is also used for analytics and reward breakdowns across a session.

4.3 Session Overview

A user can leave and log on to saved sessions which all operate in separate environments and access separate models, to avoid leakage across sessions. All of the user’s interactions including the prompts, responses generated by the model, individual trait scores, scalar rewards, and timestamps are recorded into a JSONL file which can be used for statistical analysis later on.

LoRA adapter weights are saved manually by the user, which saves the current adapter weights and tokenizer into a file stored in the session’s directory. Since only the adapter weights are saved and not the entirety of the base model, these checkpoints are approximately 80MB, regardless of the model’s size. Upon startup, if such a file exists, then it gets loaded into the model, restoring the training state of the checkpoint without reloading the base model. Training history is tracked by appending timestamps, loss, status, and step count for each GRPO update step. The user can export a trained model as either a 80MB zip file containing the adapter weights that can be then loaded in with PEFT, or the fully merged 3GB model that doesn’t need to be loaded in with a PEFT dependency.

4.4 VRAM Budgeting

The primary hardware constraint of this system is GPU memory, or VRAM, which limits both the size of the model that can be loaded and the memory available for the forward and backward passes during training. Initial development used Qwen2.5-3B-Instruct, however the 3B model exceeded the available VRAM budget and the system was switched to the 1.5B version to cut parameters. To further reduce memory consumption, LoRA is used for fine-tuning, the base model weights are frozen entirely and only a small set of low-rank adapter matrices are trained, reducing the number of active parameters from 1.5 billion to approximately 4.4 million. Additionally, by utilizing 4-bit quantization with the BitsAndBytes library I reduce the precision of the stored weight values from the standard float32 (4 bytes per parameter) or bfloat16 (2 bytes per parameter) down to a 4-bit integer representation, which cutting the memory the base model weights by approximately 4 times compared to initial bfloat16. Altogether these techniques make the system deployable on a consumer GPU with as little as 4GB of VRAM. Table ?? showcases the breakdown of VRAM allocation during a training step.

5 GRPO

5.1 Background

Group Relative Policy Optimization (GRPO) is a reinforcement learning algorithm designed to better the reasoning for LLMs. It eliminates the need for a separate value network by computing advantages relative to a group of completions for the same prompt. It optimizes the performance by generating G responses for a single prompt, scores rewards, and normalizes the rewards within the group to produce advantages, which are then used to update the model during the training step. This pipeline makes adding a human in the loop easily integratable and well suited since the rewards are sparse and come directly from user

feedback, rather than an automated function. The system implements a custom online GRPO algorithm that runs automatically after every three iterations of response grading, which can be modified manually through code. The online step allows the model to improve instantaneously as the user provides feedback.

5.2 Algorithm

The online training step implements a custom version of GPRO designed for single-sample, human-in-the-loop updates. The step is implemented as a standalone function that receives pre-scored response groups directly from the UI and performs one gradient update per iteration.

- **Input and grouping:** the step operates on a list of groups, each containing a prompt and a set of rewards graded by the human. These rewards are produced by the reward function seen in the Reward System Section.
- **Advantage normalization:** for each group, rewards are normalized into advantages where μ and σ are the mean and standard deviation respectively, and the $\epsilon = 1e-8$ value prevents a division by zero. Normalization is relative to the group, and all of the completed logged rewards are judged against those of the same prompt, not a global baseline.

$$\text{advantages} = (\text{rewards} - \text{mean}(\text{rewards})) / (\text{std}(\text{rewards}) + 1e-8)$$

$$a_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon} \quad (2)$$

- **Log probability computation:** for each completed iteration, the model computes the sequence log-probability which is the sum of per-token over all of the completed reward tokens conditioned to the prompt. The prompt and the completed jsonl rewards are concatenated onto a sequence and passed into the model in one forward pass; the resulting logits are then sliced to cover all of the completed tokens. Log-softmax is applied and the log-probability of each actual token is extracted and summed. These log probabilities are used in two different ways for the loss. The policy gradient term flows through the gradients through to make high-advantage completions more likely under the updated policy, and the difference $\log P_\pi - \log P_{\text{ref}}$ measures drift from the base model, which acts as a regularizer.

$$\log P(\text{completion} \mid \text{prompt}) = \sum_i \log P(t_i \mid \text{prompt} + \text{completion}_{1:i}) \quad (3)$$

- **Policy and reference forward passes:** each completion requires two sequential forward passes on the same model instance. The policy pass runs with LoRA adapters active and the gradients enabled, which produces the $\log P_\pi$. The reference pass disables the adapter layers and wraps the forward pass in `torch.no_grad`, which produces $\log P_{\text{ref}}$ from the frozen base weights with no gradient allocation. The adapters are re-enabled right after the reference pass. The KL divergence, which is a measure of how far the update policy has drifted from the base model.

The divergence between the policy and the reference is approximated as an estimate for a single sample with: $\text{KL} \approx \log P_\pi - \log P_{\text{ref}}$. If the number is large and positive, the policy has drifted toward generating this kind of text. If it's near zero, the policy still resembles the base model for this completion. Though it's a slightly noisy estimate, it is cheap and is informative for the one sample.

- **Loss Function:** The per completion loss combines the policy gradient term and the KL penalty:

$$\mathcal{L} = -a_i \cdot \log P_\pi + \beta \cdot \text{KL} \quad (4)$$

where $\beta = 0.1$. The negative advantage term increases the log-probability of the high-advantage completions and decreases those of the low-advantage completions. Loss terms are accumulated across all completions and all groups, averaged into one scalar, and one backward and optimizer step is called.

5.3 Training Setup

The optimizer is AdamW [5] over the LoRA parameters only, with a learning rate of 10^{-5} . The optimizer is initialized upon loading the system. Momentum is carried through the training session and accumulates continuously, and each gradient builds on the history of all prior updates.

The training step executes a background thread, and disables the response generation while running. Since the LoRA weights are updated in place, the next iteration immediately uses the updated model without the need to reload the system or execute a manual user call.

6 USER INTERFACE

6.1 Overview

The system is built with *Streamlit* and organized into six different pages that we can navigate to using the sidebar. The sidebar allows a user to create sessions, and also contains the session switcher, which allows the user to move between experiments without restarting the application, and also reboot from a saved checkpoint if it exists, since selecting a session loads up a saved state in memory from the JSONL files. The interface is designed such that the entire RLHF workflow including defining the reward criteria, browsing the dataset, generating responses, scoring, and training, can be completed without writing a single piece of code.

6.2 Setup

The setup page is where the user defines the reward traits that guides the fine tuning. Each trait is given a name, description, and a weight. Traits can be added, removed, or reweighted at any time, or during a session, though it is not advised as it may introduce unwanted noise. These traits are automatically added into the Grade page and used as scoring sliders and for the reward computations. Since the traits are defined at runtime, the same system can be used to optimize for different behavioural objectives across different sessions, without editing the backend code.

6.3 Prompt

The prompt page handles the dataset browsing and the user defined instruction that would be used for the prompt construction. The user selects a category from the 20 Newsgroups corpus and browses the individual posts, there is also an option to randomly select an article from a set instead of browsing through one by one. The user is also able to filter posts by minimum and maximum character length before being displayed, making it easier to navigate. Selecting a post sends the text to an article text box, in which the user can do more processing (i.e removing trailing characters or citations at the end of an article). The user then writes or edits an instruction describing what the model should do with the article. Instructions are automatically saved as a preset and restored on the next iteration so it does not have to be retyped after each training session.

6.4 Grade

The grade page is where the user will be scoring the responses generated by the model, and also where the training is initialized. The user chooses to generate either two or four responses and sets a target response length using a slider. The target word count is appended to the generation prompt as a "soft" instruction, which guides the model and prevents premature truncation. Generating the responses calls the model and populates the response fields automatically.

Each response is graded independently of each other but can be used as comparisons. For every trait defined at the Setup page, the user rates the response by using a slider that goes between -5 and $+5$. The scalar reward is computed in real time as the sliders are adjusted, showing both the individual per trait contributions and the final scalar value.

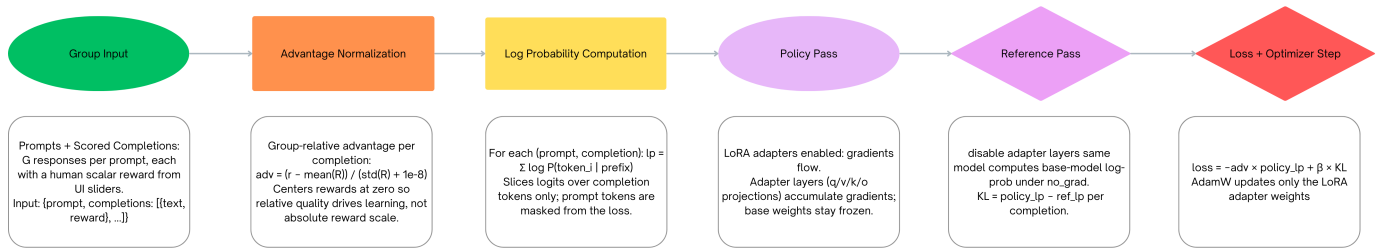


Fig. 3: Custom online GRPO training step. For each scored group, advantages are normalized within the group, policy and reference log-probabilities are computed on a single shared model, and one AdamW step updates the LoRA weights in place.

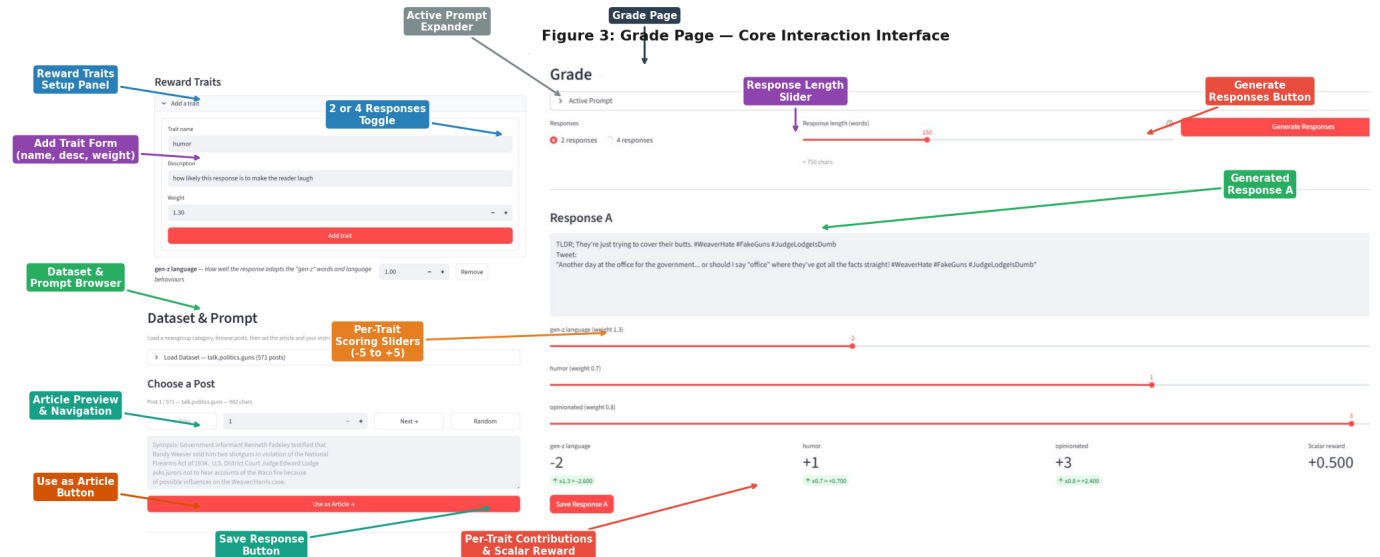


Fig. 4: **Grade Page Interface.** The core interaction page where generation, scoring, and online training converge. The left sidebar handles trait configuration and dataset browsing. The right panel displays generated responses with independent per-trait sliders and a real-time scalar reward.

Once the user is satisfied with the scoring, saving the response appends it into a JSONL file that is stored to the interaction log used for training later on.

Saving all of the responses for a given prompt completes a round, which is added to the grading buffer. After three complete rounds, the GRPO step is run automatically as a background thread. A banner pops up and informs the user that the background thread is running and also informs the user once this thread has been completed. While the background thread is running, the model cannot be accessed to generate a response to avoid concurrent access. Once the step is completed, the model is live with the updated weights already in place, with no needed user action.

6.5 History

The history page provides an overview of all of the saved interactions in the current session. The left panel lists every entry by timestamp and scalar reward. Selecting an entry opens a panel on the right that shows the full prompt, response, per-trait scores, weighted contributions, and final scalar reward for that interaction. This section allows the user to review past grading decisions and track the evolution of their interactions over a session.

6.6 Analytics

The Analytics page visualizes the reward signal across the session, and has three tabs. The Reward Trend tab shows a line chart of scalar reward over time, making it easy to see whether the model's outputs are improving across rounds. The Trait Breakdown tab shows a bar chart of average score per trait across all saved interactions, showing which of the traits the user has been rewarding or penalizing most consistently. Lastly, the Statistics tab shows summary metrics: total

graded responses, average reward, and maximum and minimum reward recorded in the session.

6.7 Session

The Session page mostly manages the exporting of the model. It also displays the session level statistics such as total graded responses, average reward, and number of completed online training steps, and the full online training log showing step number and loss for each update. Save Session writes the current LoRA adapter weights to the session's checkpoint directory; these weights are automatically reloaded the next time the app starts with this session active. Load Saved Weights restores a previously saved checkpoint manually without restarting the app.

Once a checkpoint has been saved, two export options become available. The first downloads the LoRA adapter files as a ZIP archive of approximately 80MB, which can be loaded on top of the base model using PEFT. The second option merges the LoRA weights directly into the base model weights and saves a standalone model of approximately 3GB that requires no PEFT dependency to load. Exporting the model is done by the user, nothing is merged or exported automatically by the system.

7 USE CASES

7.1 Tone and Style Fine-Tuning

The primary use case for the system is fine tuning a large language model toward a specific tone or conversational style through a series of direct human feedback rewards. A user begins by defining a set of reward traits on the set up page: for example, humor, directness, and opinionated, each being assigned a weight that reflects its importance to the final reward scalar. The user then browses the 20 Newsgroups dataset on the Prompt page, selects an article, and writes an instruction

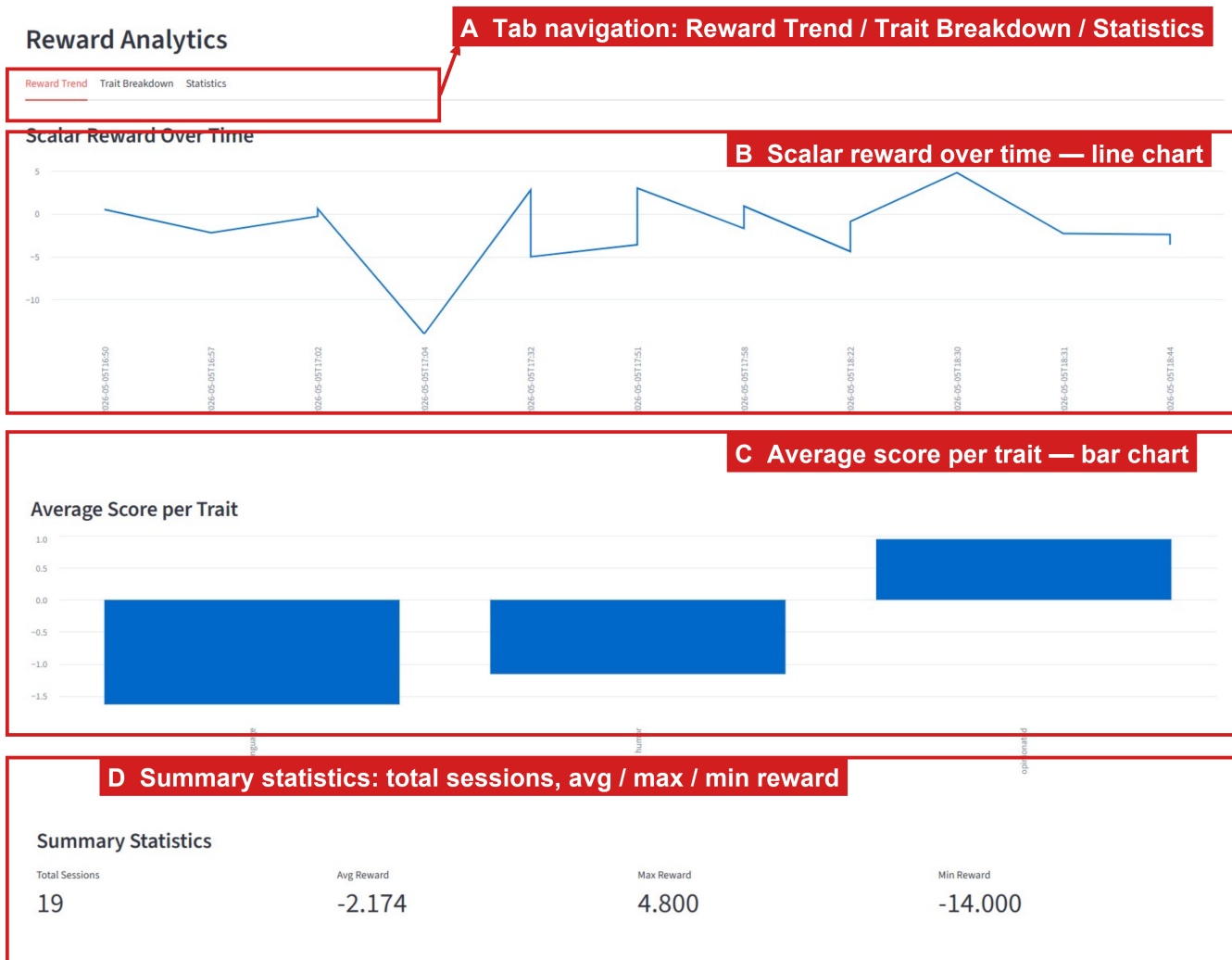


Fig. 5: Analytics page showing the three-tab reward dashboard. The *Reward Trend* tab (shown) plots scalar reward over interactions as a time-series line chart. The *Trait Breakdown* tab shows per-trait average scores as a bar chart. The *Statistics* tab displays session-level summary metrics including total graded responses, average reward, and maximum and minimum reward.

describing the desired behavior of the model. On the Grade page, the model generates two to four responses, which the user scores independently on each trait. After three complete rounds the model updates automatically, and the next generation reflects the refined weights.

The user’s scoring decisions are the only signal the model receives. Over multiple sessions the model shifts toward the behavioral profile the user has defined. The Analytics page provides a reward trend that makes the direction of change visible across rounds. This approach can be generalized to whichever set of behavioural traits that the user defines, optimizing toward the user’s preferences.

7.2 Behaviour Suppression

The reward system supports negative scoring, which makes the system equally capable of suppressing unwanted behaviors. For example, a slider score of -5 on a trait applies a strong penalty to that behavior, pushing the model away from it across subsequent training steps. It uses the same mechanism and underlying system as the process for pushing towards a desired model behaviour, and just exploits the natural design of GRPO and reinforcement learning.

In practice this means a user can target specific behaviors for elimination without defining a separate objective. Consistently scoring responses negatively steers the model away from the given responses. Since each response is scored independently per trait, the user can

simultaneously reward one dimension while penalizing another within the same round, giving fine-grained control over the direction of fine-tuning. Though it is more effective to lean towards either extreme, since scoring behaviour may cancel out in the summed reward scalar function.

7.3 Multi-Session Experimentation

Since sessions run in isolation, it allows a user to run parallel fine-tuning experiments on the same base model without leaking information into other sessions. Each session maintains its own interaction log, LoRA checkpoint, and reward history. For example, a researcher can run one session optimizing for “Shakespearean language” and another optimizing for “Short sentences”, compare reward trends on the Analytics page across sessions, and export the resulting models independently for evaluation.

This makes the system useful since a user who is unsure which trait configuration produces the best results can test multiple configurations across sessions and use the reward history as a basis for comparison.

7.4 Low-Resource Deployment

All of the use cases above run on a consumer GPU with as little as 4GB of VRAM. Since most interactive RLHF research relies on access to cloud compute or multi-GPU infrastructure, constraining the system to



Fig. 6: Scalar Reward Over Time: General Traits. Reward signal across 12 interactions when fine tuning toward broad objectives (warmth, clarity, relevance). The base model already satisfies these traits at baseline, producing consistently high scores with no upward trend.

a small GPU allows individual researchers working on local hardware to experiment with small RLHF systems to fine tune compact models. They can run the full generate-score-train loop locally and export and locally run the trained model on any hardware that can run the base model.

8 DISCUSSION

8.1 What Worked

There were several design choices made that worked extremely well in practice. The single-model KL reference trick mentioned in Section 4.3, which consisted of disabling some adapter layers to simulate the frozen reference model on the same model instance, removed the need for storing a second model copy without having much impact on training. This was one of the biggest changes to the original GRPO methodology, which was done to optimize the memory usage in the system, and worked cleanly combined with the LoRA architecture without requiring any modifications to how the loss is computed.

The three round buffer worked as a practical balance between feedback latency and training stability. Three rounds provided enough grouped completions for the advantage normalization to be useful while keeping the wait between model updates relatively short enough that the user can witness the effects of their scoring in a single sitting. The session isolation and checkpoints proved to be one of the most effective since the user can easily pick up where they left off after exiting the system without worrying about whether their progress was lost or whether their training in one session bled into another separate session, allowing users to also test different blends of trait configurations.

8.2 Results

Reward trends weren't completely straightforward and varied depending on the specificity of the user defined reward traits and the user's objective in fine tuning the model. When the traits were defined broadly and leaning towards more general qualities (i.e. clarity, warmth, relevance), the rewards did not show an upward trend or show a consistent increase over time. This is largely due to the base model being good enough to reflect the instructions given by the user well enough, and as a result, has a baseline response that scores reasonably well (according to the user) without any fine-tuning. When the base model is already close to satisfying the user's objective, there is not much to be updated on the gradients and the model has limited room for improvement.

On the other hand, rewards showed a clearer upward trend when the fine-tuning objective was specific enough that the base model didn't satisfy it. A very particular tone, a strange way of phrasing, incredibly strong opinions, or irregular structural humor patterns, that the base model could not capture in the beginning. In these cases, the advantage normalization had a lot more to work with since the gap between high-scoring and low-scoring completions within a certain group was larger, which produced stronger and larger gradient updates. This findings shows that the system is most effective when the user's objective is specific rather than general, and when the base model's default response generation leaves gaps for meaningful changes.

8.3 Limitations

The most significant limitation is that the system relies on consistent reward scoring by the user to make sure the quality of the rewards are helping the model learn rather than hindering its progress. Inconsistent



Fig. 7: Scalar Reward Over Time: Specific Traits. Reward signal across 12 interactions when fine tuning toward a specific behavioral objective (gen-z language, humor, opinionated), base model does not satisfy these traits. we can see the performance increasing slightly over time.

scoring across multiple rounds of iterations produce noisy gradients that destabilize the training and diverge the intended behavioral direction. The system does not have a mechanism in place that can flag or detect inconsistencies in scoring, and there is no aggregation that can smooth it out.

The 1.5B base model also has a limit to how much it can be fine tuned. Since the model is also tuned to follow instructions, there is less room for more drastic behavioural shifts, and a small number of online training steps is limited. Moreover, the three round buffer size in between iterations is hard coded so the user cannot specify whether they want more frequent or more stable updates, though this can be added in future work.

Lastly, there is no set way to evaluate the system performance. We can analyze the reward trends and see behaviours through response generation but it is difficult to assess whether the behavioural changes are generalizing to new inputs or specific to the articles it has seen during the grading process.

8.4 Future Work

The system is quite minimal meaning there is a lot of extension that can be made. As already mentioned earlier, allowing the user to modify the buffer size and training frequency would give the user direct control over the tradeoff between feedback latency and gradient stability. Supporting different datasets (i.e. twitter threads or reddit subreddits) to broaden the training process and possible final objectives would make the system more flexible to different instruction prompts and also its usage. Lastly, integrating a larger base model or one that is not yet harshly tuned would allow the user to see bigger effects with fine tuning.

9 CLOSING THOUGHTS

Overall the system is a skeleton and can be improved and extended in various ways. I was able to gain a lot of insights on GRPO algorithms and test out how different rewards would affect subsequent responses generated by the model. Experimenting with these models also gave me a viewpoint on the difficulty of rewriting a tuned model and how well these supposedly smaller models produce responses. I plan on extending the system to reddit's AITA subreddit so see if we can produce responses that would be closer to human behaviours and train different configurations of traits to see whether we could develop very specific personalities and response behaviours.

REFERENCES

- [1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, vol. 30, 2017. 1
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36, 2023. 1, 2
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2
- [4] K. Lang. Newsweeder: Learning to filter netnews. In *Proceedings of the 12th International Conference on Machine Learning (ICML)*, pp. 331–339, 1995. 1
- [5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 4

- [6] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. 35:27730–27744, 2022. [2](#)
- [7] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. [1](#)
- [8] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. [2](#)
- [9] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Hu. TRL: Transformer reinforcement learning, 2020. [2](#)